

Different Data Formats and Why We Need Them and XML notes.

Every app uses data.

But one important question is:

Why do different systems use different data formats?

Why not use just one format everywhere?

Because every system has different needs.

Some systems want:

- human readability
- fast communication
- smaller data size
- strict validation
- better performance
- easier parsing
- compatibility between different technologies

That is why different data formats were created over time.

Formats like:

- XML
- JSON
- BSON
- YAML
- TOML

all came to solve different problems.

And the story starts with XML.

XML

XML stands for Extensible Markup Language.

XML was created by the [W3C organization](#) to make data sharing easier across different systems.

At that time, internet was growing very fast.

Different applications were built using different languages and technologies.

One system might use Java.

Another might use C++.

Another might use .NET.

But all of them needed to exchange data.

That created a huge problem.

There was no standard way to structure and transfer data.

HTML already existed, but HTML was mainly designed to display data on webpages, not to describe data.

So XML was created to solve this.

According to the official [W3C XML Specification](#), XML was designed for:

- simplicity
- usability over the internet
- storing and transporting structured data

What Makes XML Special?

The biggest feature of XML was flexibility.

You could create your own tags.

Example:

```
<user>
  <name>Chandan</name>
  <age>27</age>
</user>
```

Here:

- <user>
- <name>
- <age>

are custom tags.

XML does not force fixed tag names.

That is why it is called "Extensible".

You can extend it according to your own data structure.

XML vs HTML

Many people confuse XML and HTML.

But both were built for different purposes.

HTML focuses on displaying data.

Example:

```
<h1>Chandan</h1>
```

XML focuses on describing data.

Example:

```
<name>Chandan</name>
```

HTML says:

"How should this look on screen?"

XML says:

"What is this data?"

This difference was extremely important for software systems.

Why XML Became So Popular

XML became very popular because it solved many internet problems.

It was:

- human readable
- machine readable
- platform independent
- language independent
- structured

A Java server and a .NET application could exchange XML easily.

That was revolutionary at that time.

XML Structure

XML stores data in a hierarchical structure.

It works like a tree.

Example:

```
<company>
  <employee>
    <name>Chandan</name>
  </employee>
</company>
```

Tree structure:

```
company
├── employee
│   └── name
```

This made XML powerful for representing complex relationships.

XML Rules

XML is strict.

That strictness helped systems validate data correctly.

Every tag must close

Correct:

`<name>Chandan</name>`

Wrong:

`<name>Chandan`

Tags must be properly nested

Correct:

```
<a>
  <b></b>
</a>
```

Wrong:

```
<a>
  <b>
</a>
</b>
```

One root element is required

Correct:

```
<data>
</data>
```

Wrong:

```
<a></a>
<b></b>
```

XML Validation

One powerful feature of XML was validation.

Systems could define rules like:

- age must be a number
- email is required
- salary cannot be negative

This was done using:

- DTD
- XSD (XML Schema)

That made XML very useful for enterprise systems and banking systems where data correctness was critical.

Where XML Was Used

XML became the backbone of many enterprise systems.

Used in:

- SOAP APIs
- banking systems
- Android layouts
- Maven configuration (`pom.xml`)
- web services
- configuration files
- document systems

Even today XML still exists in many large systems.

Then XML Started Creating Problems

As the internet evolved, web applications became faster and lighter.

Mobile apps started growing.

APIs started handling massive traffic.

And XML slowly started becoming problematic.

Problem 1: Too Much Extra Data

XML had lots of repeated tags.

Example:

```
<name>Chandan</name>
```

The opening and closing tags increased payload size.

Large XML files consumed:

- more bandwidth
- more storage
- more network cost

This became expensive at scale.

Problem 2: Parsing Was Heavy

XML parsers were complex.

Processing XML required more memory and CPU.

For modern applications needing speed, this became a bottleneck.

Problem 3: Large Files Became Hard to Read

Small XML files looked clean.

But deeply nested XML became messy very quickly.

Example:

```
<a>
  <b>
    <c>
      <d>
```

Large enterprise XML files became difficult to debug and maintain.

Problem 4: Slower APIs

Web and mobile applications needed fast communication.

But XML payloads were large.

That increased:

- API response size
- network latency
- parsing time

Developers wanted something lighter and faster.

Then JSON Arrived

Developers started searching for a simpler data format.

Something:

- lightweight
- faster
- easier to parse
- easier for JavaScript
- better for web APIs

That is where JSON entered.

JSON removed much of the extra text used in XML.

It became smaller, cleaner, and faster.

And slowly JSON started dominating modern APIs.

That completely changed how applications communicate over the internet.

Simple One-Line Summary

XML solved the internet's data sharing problem, but its complexity and heavy structure created the need for JSON.